

Weekly Report Generator & Team Dashboard

Developer Implementation Guide

Stack: Spring Boot • React + TypeScript • MySQL • LLM-powered AI Assistant

Optimized consolidated guide — architecture, database, workflow, APIs, and build order

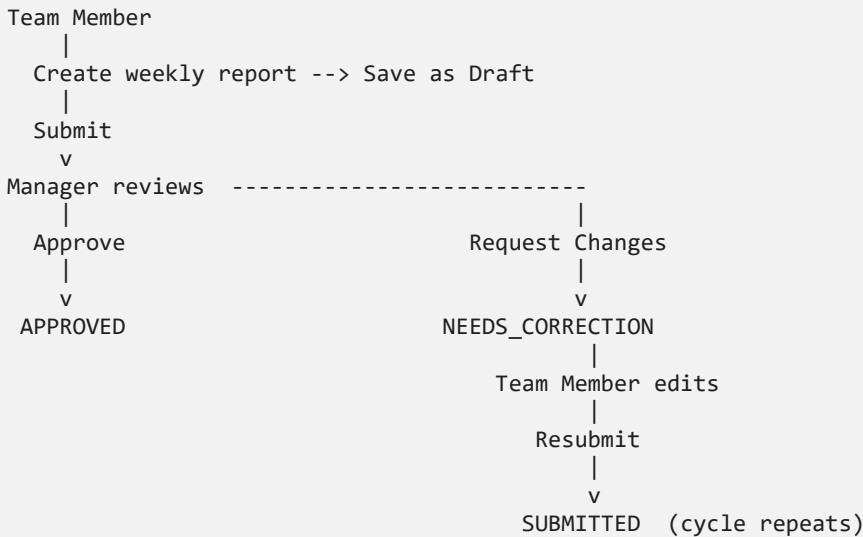
Contents

1. Objective & Core Concept
2. Technology Stack
3. System Architecture
4. Repository & Folder Structure
5. Database Design (MySQL)
6. Critical Business Rules
7. Authentication & JWT Flow
8. REST API Reference
9. Report Status State Machine
10. Frontend Pages (7+ required)
11. Manager Dashboard
12. AI Chat Assistant (Good-to-Have)
13. Validation, Testing & Security
14. Seed Data, Docker & Environment
15. Recommended Build Order
16. Demo Video Script
17. Final Deliverables Checklist

1. Objective & Core Concept

Build a full-stack internal tool where Team Members submit structured weekly work reports, Managers review and either approve or send them back for correction, and Managers get a consolidated dashboard across the whole team. The review/correction cycle is the heart of this assignment — everything else supports it.

සාරාංශය: Core requirement එක — Team Member report එකක් හඳුනාලා → Submit කරනවා → Manager Approve කරනවා හෝ Request Changes කරනවා → Correction නම් Edit කරලා Resubmit කරනවා → අවසානයේ Approved.



2. Technology Stack

Frontend

- React 18 + TypeScript
- Vite (build tool)
- Material UI (or Tailwind, if you prefer utility CSS)
- React Router v6
- Axios (API client with interceptors for JWT)
- React Hook Form + Zod (form + schema validation)
- Recharts (dashboard charts)

Backend

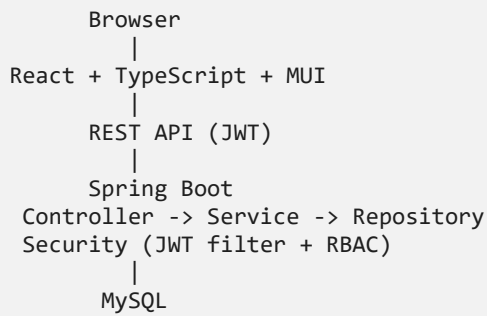
- Java 17
- Spring Boot 3.x
- Spring Security + JWT (stateless auth)
- Spring Data JPA + Hibernate
- Bean Validation (jakarta.validation)
- MySQL 8
- Flyway (schema migrations)

AI (Optional / Good-to-Have)

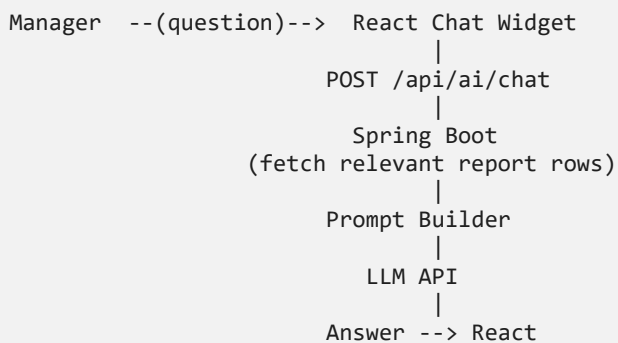
- Any LLM provider (Anthropic Claude, OpenAI, or an open-source model) called server-side only

AI feature එක mandatory නෙවෙයි — bonus points එකරයි. Core workflow එක complete කරන එකට priority දෙන.

3. System Architecture



With the AI assistant layered in:



Key decision: the AI call is made from the backend, never directly from React. This keeps the API key secret, lets you enforce role checks before answering, and lets you inject only the report fields that are actually relevant instead of giving the model open access to the database.

4. Repository & Folder Structure

Two repositories is the cleanest approach for this assignment:

- weekly-report-dashboard-backend
- weekly-report-dashboard-frontend

Backend (Spring Boot, Maven)

```
backend/src/main/java/com/yourorg/weeklyreport/
config/      SecurityConfig, CorsConfig, OpenApiConfig
auth/        AuthController, JwtService, RegisterRequest, LoginRequest
user/        User, Role, UserController, UserService, UserRepository
project/     Project, ProjectController, ProjectService, ProjectRepository
report/
  entity/    WeeklyReport, ReportVersion, TaskEntry, Blocker, Achievement, HoursEntry
  dto/       ReportRequest, ReportResponse, ReportSummaryDto
  ReportController.java, ReportService.java, ReportRepository.java
review/      ReviewAction, ReviewController, ReviewService
dashboard/   DashboardController, DashboardService (aggregation queries)
ai/          AiController, AiAssistantService, PromptBuilder, LlmClient
exception/   GlobalExceptionHandler, custom exceptions
mapper/     entity <-> DTO mappers
```

Strict layering: Controller never contains business logic — it delegates to Service, which uses Repository.

Controller -> Service -> Repository -> MySQL

Frontend (React + TypeScript)

```
frontend/src/  
  api/          axios instance, authApi, reportApi, dashboardApi, aiApi  
  components/   common/, reports/, dashboard/, projects/  
  layouts/      AppLayout, AuthLayout  
  pages/  
    auth/       LoginPage, RegisterPage  
    reports/    MyReportPage, ReportHistoryPage, ReportDetailPage, ReportEditPage  
    manager/    TeamDashboardPage, ManagerReviewPage, MemberProfilePage  
    projects/   ProjectsPage  
    users/      UserManagementPage  
  hooks/, context/, types/, utils/, routes/
```

5. Database Design (MySQL)

Database name: weekly_report_db. This schema is normalized with a proper roles table (rather than a hardcoded enum) so Admin/Manager/Team Member can be managed from the User Management page without a code change.

Entity Relationship Overview



roles

Field	Description
id	PK
name	TEAM_MEMBER / MANAGER / ADMIN

users

Field	Description
id	PK
full_name	VARCHAR(100)
email	UNIQUE, VARCHAR(150)
password_hash	BCrypt hash — never store plain text
role_id	FK -> roles.id
is_active	BOOLEAN, for deactivating leavers
created_at / updated_at	DATETIME

projects

Field	Description
id	PK
name	e.g. Client A, Internal Tooling, R&D, Marketing
description	VARCHAR(255)
is_active	soft-delete flag instead of hard DELETE

user_projects (optional assignment table)

Field	Description
user_id, project_id	composite PK, both FK

weekly_reports

Field	Description
id	PK
user_id	FK -> users.id (owner)
project_id	FK -> projects.id
week_start / week_end	DATE
status	DRAFT / SUBMITTED / NEEDS_CORRECTION / APPROVED
tasks_planned_next_week	TEXT
notes	TEXT (optional notes/links)
current_version_no	INT, increments on each resubmit
created_at / updated_at	DATETIME

UNIQUE(user_id, week_start) constraint එකක් දාන්න — user කෙනෙක්ට එකම week එකට report දෙකක් හදන්න බැරි වෙන්න ඕනේ, සහ 'not yet started' status එක mathematically derive කරන්න පුළුවන් වෙනවා (row එකක් නැත්නම් = not started).

task_entries

Field	Description
id	PK
report_id	FK -> weekly_reports.id
task_name	VARCHAR(255)
priority	LOW / MEDIUM / HIGH
planned_pct / actual_pct	INT 0–100
status	NOT_STARTED / IN_PROGRESS / DONE / BLOCKED
time_planned_hrs / time_spent_hrs	DECIMAL(5,2)
output_deliverable	VARCHAR(255)

blockers / achievements

Field	Description
id	PK
report_id	FK -> weekly_reports.id
description	TEXT
is_key_issue / is_key_achievement	BOOLEAN — only one should be true per report (enforce in service layer)

hours_breakdown (optional feature)

Field	Description
id	PK
report_id	FK -> weekly_reports.id
task_type	DEVELOPMENT / TESTING / MEETINGS / DOCUMENTATION / OTHER
hours	DECIMAL(5,2)

review_actions

Field	Description
id	PK
report_id	FK -> weekly_reports.id
reviewer_id	FK -> users.id (the manager)
action	APPROVED / CHANGES_REQUESTED
comment	TEXT — required when requesting changes
against_version_no	which version of the report this comment refers to
created_at	DATETIME

report_versions (bonus: version history)

Field	Description
id	PK
report_id	FK -> weekly_reports.id
version_no	INT, increments each submit
content_json	JSON snapshot of tasks/blockers/achievements/hours at submit time
submitted_at	DATETIME

Correction cycle එකේදී old content overwrite කරන්න එපා — status SUBMITTED වෙනකොට content_json snapshot එකක් report_versions වලට save කරන්න. review_actions.against_version_no කියන field එකෙන් manager comment එක කුමන version එකට කරපු එකක්ද කියලා link කරගන්න පුළුවන්.

6. Critical Business Rules (enforce server-side)

The UI can hide buttons, but every one of these rules must also be enforced in the Spring Boot service layer — the live-coding round and the evaluation both specifically test this.

Rule 1 — Ownership

A Team Member can only view/edit their own reports. Any request for another user's report must return 403, not an empty result.

Rule 2 — Manager visibility

A Manager can view every team member's reports, but cannot edit report content — only status and comment fields, via the review endpoint.

Rule 3 — Editable states

A report is only editable by its owner while status is DRAFT or NEEDS_CORRECTION. SUBMITTED and APPROVED are read-only for the owner.

Rule 4 — Valid transitions only

```
Allowed:   DRAFT -> SUBMITTED
           SUBMITTED -> APPROVED
           SUBMITTED -> NEEDS_CORRECTION
           NEEDS_CORRECTION -> SUBMITTED

Rejected:  APPROVED -> DRAFT
           DRAFT -> APPROVED (must go through SUBMITTED)
           any transition triggered by someone who isn't the owner/manager
```

Example service-layer guard:

```
public void submit(Long reportId, Long requesterId) {
    WeeklyReport report = getOwnedReport(reportId, requesterId); // 403 if not owner
    if (report.getStatus() != Status.DRAFT
        && report.getStatus() != Status.NEEDS_CORRECTION) {
        throw new IllegalStateException(
            "Cannot submit from status " + report.getStatus());
    }
    snapshotVersion(report);
    report.setStatus(Status.SUBMITTED);
    reportRepository.save(report);
}
```

7. Authentication & JWT Flow

Register

```
React -> POST /api/auth/register -> validate -> BCrypt hash -> save user -> MySQL
```

Login

```
React -> POST /api/auth/login -> Spring Security validates -> generate JWT -> return to React
```

Every protected request

```
React sends: Authorization: Bearer <JWT>
              |
JwtAuthenticationFilter validates token
```

```
|  
Extract userId + role -> SecurityContext  
|  
Controller (protected by @PreAuthorize)
```

Logout with a stateless JWT is primarily client-side (discard the token). If you want server-side revocation (e.g. for a 'log out everywhere' feature), add a short-lived token blocklist or a token version column on the user — not required for this assignment, but worth mentioning in the presentation as a future improvement.

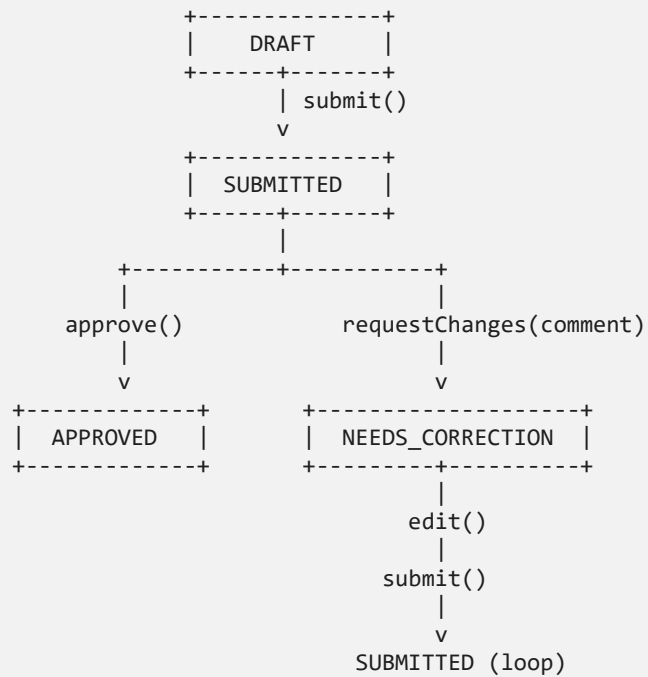
8. REST API Reference

Method	Endpoint	Role	Purpose
POST	/api/auth/register	public	Create account (default role TEAM_MEMBER)
POST	/api/auth/login	public	Returns JWT
GET	/api/auth/me	any	Current user profile
GET	/api/users	MANAGER/ADMIN	List team members
POST	/api/users/invite	ADMIN	Invite a new user
PUT	/api/users/{id}/role	ADMIN	Change role
PUT	/api/users/{id}/status	ADMIN	Activate/deactivate
GET/POST/PUT/DELETE	/api/projects	MANAGER (write) / any (read)	Project CRUD
POST	/api/reports	TEAM_MEMBER	Create draft report for a week
PUT	/api/reports/{id}	TEAM_MEMBER (owner)	Edit while DRAFT/NEEDS_CORRECTION
GET	/api/reports/{id}	owner or MANAGER	Report detail
GET	/api/reports/my?status=&page=	TEAM_MEMBER	Own report history, paginated
POST	/api/reports/{id}/submit	TEAM_MEMBER (owner)	Draft/NeedsCorrection to Submitted
GET	/api/reports/{id}/versions	owner or MANAGER	Version history list
GET	/api/manager/reports?week=&userId=&projectId=&status=&page=	MANAGER	Filtered, paginated team list
POST	/api/manager/reports/{id}/approve	MANAGER	Approve
POST	/api/manager/reports/{id}/request-changes	MANAGER	body: { comment }
GET	/api/manager/dashboard/summary?week=	MANAGER	Compliance rate, correction count, open blockers
GET	/api/manager/dashboard/tasks-trend	MANAGER	Chart data
GET	/api/manager/dashboard/status	MANAGER	Status by team member
GET	/api/manager/dashboard/projects	MANAGER	Workload by project
GET	/api/manager/dashboard/time-distribution	MANAGER	Time by task type
GET	/api/manager/dashboard/activity	MANAGER	Recent activity feed
POST	/api/ai/chat	MANAGER	AI Q&A over team reports

Method	Endpoint	Role	Purpose
POST	/api/ai/team-summary?week=	MANAGER	AI-generated team summary

Every endpoint returning a list must support pagination and/or filtering — this is called out explicitly in the assignment and is an easy point to lose.

9. Report Status State Machine



10. Frontend Pages (7+ required)

The assignment requires at least 7 of the listed page types, connected to real backend data. The route list below comfortably exceeds that and maps directly to the required list.

Route	Page	Satisfies requirement
/login, /register	Auth pages	Login / Register page
/reports/new, /reports/:id/edit	Create / edit report	Personal weekly report page
/reports/history	Report history	Report history page
/reports/:id	Report detail (read-only)	Report detail / view page
/team/:id	Member profile (manager view)	Team member profile page
/projects	Project/category CRUD	Project / category management page
/users	User management (admin)	User management page
/manager/reports/:id/review	Approve / Request Changes	Manager review page
/manager/dashboard	Team dashboard + charts	Dashboard & visual insights

Report form — fixed structure for every user

Weekly Report

Week Start [] Week End [] Project [dropdown]

Completed Tasks (repeatable rows)

Task Name | Priority | Planned % | Actual % | Status
| Planned Hrs | Spent Hrs | Deliverable [+ Add Task]

Tasks Planned for Next Week [text area]

Blockers [+ add] [] mark as Key Issue

Achievements [+ add] [] mark as Key Achievement

Hours Worked (optional): Development / Testing / Meetings / Documentation

Notes / Links [text area]

[Save Draft] [Submit]

Field set එක සියලුම users ලාට identical වෙන්න ඕනේ — reorder කරන්න, add කරන්න බැහැ. මේකෙන් තමයි manager dashboard එකේ reports compare කරන්න පුළුවන් වෙන්නේ.

11. Manager Dashboard

Summary cards

- Total reports submitted this week
- Submission compliance rate (submitted vs pending vs late)
- Reports currently in Needs Correction
- Open blockers across the team

Filters

- Team member

- Project / category
- Date range
- Status (including a derived 'Not Started' for members with no report row that week)

Charts (Recharts)

- Tasks completed trend over time (per person or team-wide)
- Report status by team member (stacked bar)
- Workload / task distribution by project
- Time spent by task type, team-wide
- Recent activity feed (submissions, approvals, correction requests)

Shape aggregation server-side (in DashboardService, via JPQL) and return chart-ready { label, value } arrays — don't make the frontend re-aggregate raw rows.

```
@Query("""
SELECT new com.yourorg.dashboard.ComplianceDto(
    COUNT(CASE WHEN r.status <> 'DRAFT' THEN 1 END),
    COUNT(CASE WHEN r.status = 'DRAFT' THEN 1 END))
FROM WeeklyReport r WHERE r.weekStart = :weekStart
""")
ComplianceDto getComplianceForWeek(@Param("weekStart") LocalDate weekStart);
```

12. AI Chat Assistant (Good-to-Have)

Build this after the core workflow is solid — it's bonus points, and an incomplete review cycle will cost far more than a missing AI feature.

Approach: retrieval + prompting (RAG-lite, not a vector database)

- Manager asks a question in the chat widget (e.g. 'What did the design team work on last week?')
- Backend resolves a light filter (week / member / project) from the question or from UI-selected context
- Backend queries MySQL for the matching reports and builds a compact text summary — not a raw DB dump
- That summary is injected into the prompt sent to the LLM, along with the manager's question
- The model is instructed to answer only from the given data and say when something isn't covered

```
React Chat UI
  | POST /api/ai/chat
  v
Spring Boot -> ReportService (fetch relevant reports)
  |
PromptBuilder (report data -> compact context)
  |
LLM API (Anthropic / OpenAI / open-source)
  |
Answer -> React
```

Example system prompt

You are an internal assistant for engineering managers.
Answer ONLY using the report data provided below.
If the answer isn't in the data, say so explicitly.
Do not invent names, numbers, or dates.

Reports:
{context}

Never do this

```
React --(direct call, API key exposed in browser)--> LLM Provider X
```

Data-privacy notes to include in your presentation

- /api/ai/* is manager-only, enforced with @PreAuthorize
- Only send the minimum report fields needed to answer — no password hashes, no data the requesting manager isn't already authorized to see
- Note whether your chosen LLM provider retains request data, as a documented trade-off for a take-home assignment
- Log AI queries server-side for auditability, without logging API keys

13. Validation, Testing & Security

Validation

Layer	Rules
Frontend (Zod / React Hook Form)	Required fields, email format, percentage 0-100, hours >= 0, valid week range

Layer	Rules
Backend (Bean Validation)	@NotNull, @NotBlank, @Email, @Min, @Max, @Size — mandatory, never trust the client

Required RBAC test scenarios

- Team Member A requests Team Member B's report -> expect 403
- Team Member requests a manager-only endpoint (e.g. GET /api/manager/reports) -> expect 403
- Manager attempts to edit report content directly (not via the review endpoint) -> expect 403 or 400
- Invalid status transition (e.g. APPROVED -> DRAFT) -> expect 400/409

Write these as JUnit + Spring Security Test cases (AuthenticationTest, ReportAuthorizationTest, ReportWorkflowTest, ManagerReviewTest). This is called out as 'bonus but strongly recommended' — it's a cheap way to demonstrate the RBAC design actually works, not just looks right in the code.

14. Seed Data, Docker & Environment

Seed data (required for a meaningful dashboard)

- 3-5 team members + 1-2 managers
- 3-4 projects
- Several weeks of history
- Reports spanning all four statuses (Draft, Submitted, Needs Correction, Approved), plus at least one member with a missing week to demonstrate 'Not Started'

Docker Compose (recommended, not mandatory)

```
services:
  frontend: # React build served via nginx or vite preview
  backend:  # Spring Boot jar
  mysql:    # MySQL 8, with a volume for persistence
```

Environment variables

Where	Variables
Backend	DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET, LLM_API_KEY
Frontend	VITE_API_BASE_URL

Commit a .env.example with placeholder values; never commit the real .env or API keys.

15. Recommended Build Order

Following this order means you always have something demoable, and the highest-weighted evaluation criteria (RBAC, workflow correctness) are locked in early rather than bolted on at the end.

Phase	Focus
1. Foundations	Spring Boot + MySQL + Flyway migrations; JWT auth; React skeleton + routing + AuthContext
2. Core report CRUD	Draft create/edit for the fixed-schema report (no workflow yet); My Report + Report History pages
3. Review workflow	Status transitions, review endpoint, Manager Review page; shared read-only Report Detail page
4. RBAC hardening + tests	Lock down ownership checks; write the access-control tests
5. Team Dashboard	Filters, pagination, summary cards
6. Admin pages	Projects CRUD page, User Management page
7. Charts	Recharts visual insights + aggregate endpoints
8. Version history	Snapshot on submit; versions list UI (bonus)
9. AI Assistant	Backend service first, then the chat widget (bonus)
10. Polish	Seed script, README, Docker/deployment, ER diagram export, presentation, demo video

Priority tiers

Tier	Items
Must have	Auth + JWT + RBAC; full report workflow (Draft -> Submitted -> Needs Correction -> Approved); Report History; Manager Dashboard with filters + charts; Projects CRUD; 7+ pages; REST APIs; MySQL; validation; pagination; seed data
Strongly recommended	Automated RBAC tests; report version history; Docker; deployment
Bonus	AI Chat Assistant

AI assistant එක හදන්න ගිහින් core workflow එක incomplete කරන්න එපා — AI bonus එකයි, workflow එක mandatory core requirement එකයි.

16. Demo Video Script

Camera on, face visible, and follow this exact story so every required element is covered in one continuous walkthrough:

Step	Action
1	Team Member logs in
2	Creates a weekly report, adds tasks
3	Saves as Draft
4	Submits the report
5	Logs out, Manager logs in
6	Views the Team Dashboard, applies filters
7	Opens the submitted report, clicks Request Changes with a comment
8	Logs out, Team Member logs back in
9	Sees the correction comment clearly on their report page
10	Edits the report and resubmits
11	Manager reviews again and Approves
12	Dashboard reflects the updated status
13	Show report details for 2-3 different team members to prove multi-user data
14	(If implemented) Demo the AI Chat Assistant

17. Final Deliverables Checklist

- GitHub repos: backend + frontend, with a README covering install / run frontend / run backend / run database
- ER diagram image, matching Section 5
- Google Slides presentation: architecture, DB design, key components, API/RBAC design, workflow implementation, AI approach + privacy notes, challenges faced, future improvements
- Demo video (Section 16), uploaded with the presentation and ER diagram to one shared Google Drive folder, sharing set to 'Anyone with the link'
- Submission email with the GitHub link and the Drive folder link

Links share කරන්න කලින් access permissions double-check කරන්න — link accessible නැත්නම් submission එක evaluate කරන්නේ නෑ කියලා assignment එකේම සඳහන් වෙනවා.